Vulnerability Scanning & Enumeration

Module 2 — Reconnaissance · Unit 09

You found the open doors in Unit 08. Now you learn *exactly* what's behind each one — and which are weak. Then you STOP at the plan.

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

Where we are in Module 2

- Unit 07 — Passive recon (OSINT): read public records.
- Unit 08 — Active recon & scanning: found live hosts, open ports, versions.
- **Unit 09 — Vuln scanning & enumeration** ← this week, the final unit.

> This week: turn open ports into detailed findings and a prioritized **attack plan** — then write the **Recon mini-project**.

Learning objectives (1 of 2)

By the end of this unit you can:

- **Distinguish** scanning, enumeration, and vulnerability scanning.
- **Manually enumerate** HTTP, FTP, SMB, and SSH — and explain why manual still matters.
- **Run** targeted nmap **NSE scripts** and interpret the output.
- **Describe** automated vuln scanners (Nessus, OpenVAS) at an awareness level.

Learning objectives (2 of 2)

- **Map** a service/version to a **CVE** and read its **CVSS** severity.
- **Spot** likely **false positives** and verify them manually.
- **Turn** findings into a prioritized **attack plan** — without exploiting.
- **Produce** the Module 2 **Recon mini-project report**.

Vocabulary — the three steps

Term	Meaning
Scanning	Find live hosts and open ports — "what's there."
Enumeration	Dig into a service for users/shares/versions — "what is it."
Vulnerability scanning	Check versions against known weaknesses — "what's weak."
Vulnerability	A weakness that could be exploited to harm CIA.

Vocabulary — tools & services

Term	Meaning
Service enumeration	Probing a specific service for usable detail.
SMB	Windows file/printer sharing; often enumerable.
NSE	Nmap Scripting Engine — scripts that automate checks.
Automated scanner	Nessus / OpenVAS — scans broadly, flags possible vulns.

Vocabulary — findings

Term	Meaning
CVE	Public ID for one known flaw (e.g., CVE-2011-2523).
CVSS	A 0–10 severity score (higher = more severe).
False positive	A reported finding that isn't real/exploitable.
Attack surface	All the points where a target could be attacked.
Attack plan	A prioritized list of what to try, and why.

Day 1

Scanning vs enumeration vs vuln scanning; why manual matters

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

Warm-up

> Last unit you found open **port 445 running Samba**. What's the **next** question you'd ask about it?

- What **shares** are exposed?
- What **users** exist?
- What **version** is it, and is that version weak?

That digging is **enumeration**.

Three steps that build on each other

Step	Question it answers
Scanning (Unit 08)	"What's there?" — hosts, open ports
Enumeration	"What *exactly* is it?" — users, shares, versions
Vuln scanning	"Which is *weak*?" — match to known flaws

> Each step uses the output of the last.

You can't skip a step

- You can't **enumerate** a service you haven't **found**.
- You can't **map a vuln** without knowing the service and **version**.
- Skipping ahead = guessing in the dark.

> Recon is a ladder. Each rung depends on the one below.

Why manual enumeration still matters

- Automated tools **miss context** and produce **false positives**.
- A human can spot a **misconfiguration** a scanner ignores.
- Manual enumeration finds **users and shares** a port scan never reveals.

> Tools are fast; judgment is irreplaceable. Use both.

Lab Setup + Part A — HTTP enumeration

```
curl -I http://<target-ip>/          # server header
curl http://<target-ip>/robots.txt   # disallowed paths
```

- Read the **server header** (e.g., Apache 2.2.8).
- Check **robots.txt** for paths someone tried to hide.

> Web apps (DVWA, Mutillidae, phpinfo) = a rich attack surface for Module 3.

Reading the HTTP server header

```
HTTP/1.1 200 OK
Server: Apache/2.2.8 (Ubuntu)
X-Powered-By: PHP/5.2.4
```

- `Server:` → web software + version (a CVE lead).
- `X-Powered-By:` → the app language and version.

> Two header lines, two version leads.

What robots.txt accidentally reveals

```
User-agent: *
Disallow: /admin/
Disallow: /backup/
```

- `robots.txt` asks search engines **not** to index paths.
- It does **not** hide them — it lists exactly where to look.

> A "do not enter" sign that names every interesting door.

Exit ticket — Day 1

In one sentence each, define **scanning**, **enumeration**, and **vuln scanning**.

Day 2

Manual enumeration of common services (HTTP, FTP, SMB, SSH)

Warm-up

What could an **anonymous FTP login** or an **open SMB share** leak?

- Anonymous FTP → files the owner forgot were public.
- Open SMB share → documents, backups, even credentials.
- Both can hand you **usernames** for later attacks.

FTP enumeration (port 21)

```
ftp <target-ip>      # user: anonymous   pass: (anything)
```

- Try an **anonymous login** — does it work?
- If it does, **list and browse** the files visible.
- Note the **banner/version** (e.g., `vsFTPd 2.3.4`).

> Anonymous access = an open door. What's inside is the finding.

Why anonymous FTP is so dangerous

- "Anonymous" means **no real account** is needed to log in.
- Admins enable it for convenience and **forget** what's in the folder.
- Backups, configs, and customer files routinely leak this way.

> The version is a lead; the exposed files are the finding.

SMB enumeration (ports 139/445)

```
smbclient -L //<target-ip>/ -N      # list shares (no password)
enum4linux -a <target-ip>           # shares + users + more
```

- **Shares**: e.g., `tmp`, `IPC$`.
- **User accounts**: e.g., `msfadmin`.

> The classic lesson: manual SMB enumeration finds **users and shares a port scan never could.**

Reading enum4linux output

```
[+] Share: tmp          (read/write)
[+] Share: IPC$
[+] User: msfadmin
[+] User: service
```

- **Shares** = folders you might browse.
- **Users** = login names to remember for later attacks.

> Each username is a lead; each writable share is a risk.

SSH enumeration (port 22)

```
nc <target-ip> 22      # read the banner
```

- Read the **banner/version** (e.g., `SSH-2.0-OpenSSH_4.7p1`).
- Note supported **auth methods** if shown.

> Even a "locked" service like SSH announces its version — a CVE lead.

Lab Part B — FTP, SMB, SSH

```
ftp <target-ip>
smbclient -L //<target-ip>/ -N
enum4linux -a <target-ip>
nc <target-ip> 22
```

Record: did anonymous FTP work; SMB shares/users; SSH version. Note which findings a **plain port scan would miss**.

Check your understanding

> A port scan shows `445/tcp open`. Manual SMB enumeration adds: shares `tmp` and `IPC$`, users `msfadmin` and `service`.

What did **enumeration** add that the scan alone could not?

Think before the next slide.

Answer

The scan only proved **the port was open**.

Enumeration added the **content**:

- **Share names** (`tmp`, `IPC$`) you could browse.
- **Usernames** (`msfadmin`, `service`) for later attacks.

> "Open" is the door; enumeration tells you what's behind it.

Exit ticket — Day 2

Name **one thing manual SMB enumeration can find** that a quick port scan won't.

Day 3

nmap NSE scripts + automated vuln scanners (awareness)

Warm-up

> What if nmap could run a **small program** against each open port to dig deeper?

It can. They're called **NSE scripts** — the Nmap Scripting Engine.

NSE script categories

Category	What it does
`default`	Safe, useful scripts (same as `-sC`)
`safe`	Won't crash or change the target
`vuln`	Checks for known vulnerabilities

> Pick a category, or name a specific script.

Running specific NSE scripts

```
nmap --script ftp-anon -p 21 <target-ip>
nmap --script smb-enum-shares -p 139,445 <target-ip>
```

- `ftp-anon` → "Anonymous FTP login allowed."
- `smb-enum-shares` → lists the SMB shares.

> Each script automates a check you could do by hand.

The `--script vuln` run

```
nmap --script vuln -p <ports> <target-ip>
```

- Runs **vuln-detection** scripts against the open ports.
- May flag real issues — **and false positives**.

> Compare every NSE result to your **manual** findings. Do they agree?

Automated vuln scanners (awareness)

Tool	Notes
Nessus Essentials	Free tier, registration-gated, IP-limited
OpenVAS / Greenbone	Free, fully open-source

> They flag *possible* vulnerabilities — a human still verifies.

Scanners: strengths vs weaknesses

Strength	Weakness
Broad coverage	Noisy on the network
Fast	Produces **false positives**
Thousands of checks	No human context

> Great for breadth, weak on judgment. Use them, then verify.

Lab Part C + optional D

Part C — NSE scripts:

```
nmap --script ftp-anon -p 21 <target-ip>
nmap --script smb-enum-shares -p 139,445 <target-ip>
```

Record what each reported; compare to manual findings.

Part D (optional): run Nessus/OpenVAS **or** analyze the instructor's sample report. Note top findings + CVSS; mark suspected false positives.

Check your understanding

> Your `--script vuln` run flags a "possible" vulnerability, but it also says `CONFIDENCE: low` and you can't reproduce it manually.

How should you record it?

Think before the next slide.

Answer

Record it as a **likely false positive** — a *lead*, not a fact.

- Note the scanner flagged it but **manual checks didn't confirm** it.
- Propose a verification step (check the version/config/behavior).

> A "critical" you can't reproduce is a lead, not a finding.

Exit ticket — Day 3

Give **one strength and one weakness** of an automated vuln scanner vs manual enumeration.

Day 4

Mapping findings to CVEs, false positives, and building an attack plan

Curriculum by AJ Hammond — PNPT, CRTO, OSCP, BSCP

Warm-up

You found `vsftpd 2.3.4`. How would you find out if that version is **dangerous**?

Look it up. A specific version maps to specific, public **CVEs**.

Reading a CVE

- **CVE** = Common Vulnerabilities and Exposures — a public ID for one known flaw (e.g., `CVE-2011-2523`).
- **CVSS** = a **0–10 severity score** (higher = worse).
- A CVE entry tells you: what it is, affected versions, how severe.

> Map your **starred versions** from Unit 08 to their CVEs.

What CVSS scores mean

CVSS	Severity
0.1–3.9	Low
4.0–6.9	Medium
7.0–8.9	High
9.0–10.0	Critical

A 9.8 demands attention; a 3.1 can usually wait.

Example CVE mappings (Metasploitable 2)

Service & version	CVE	What it is
vsftpd 2.3.4	CVE-2011-2523	Backdoor RCE (CVSS ~10)
Samba 3.x	CVE-2007-2447	"username map script" RCE
UnrealIRCd 3.2.8.1	CVE-2010-2075	Backdoor

Map only. **Do NOT exploit** — that's Module 3.

From version to CVE — the workflow

1. Take a **starred version** (e.g., `vsftpd 2.3.4`).
2. Search it in a CVE database or vendor advisory.
3. Read the **CVE ID**, affected versions, and **CVSS**.
4. Record it in your mapping table.

> A version string becomes a measured, citable risk.

False positives

- A **false positive** = a "vulnerability" a tool reports that **isn't real or exploitable**.
- Scanners over-report — they match patterns without context.
- **Verify manually** before trusting a finding.

> A "critical" finding you can't reproduce is a lead, not a fact.

How to verify a finding

- **Check the version** — does it actually match the CVE's affected range?
- **Check the config** — is the vulnerable feature even enabled?
- **Check the behavior** — does it respond the way the CVE describes?

> Three checks turn a scanner flag into a confident finding.

Building the attack plan (planning only)

Turn **verified** findings into a prioritized plan:

Service & version	CVE	CVSS	Confirmed?	Why it matters
vsftpd 2.3.4	CVE-2011-2523	~10	Yes	Backdoor → first target

Prioritize high-CVSS, **confirmed** findings. **No exploitation.**

How to prioritize

- **Severity first:** high CVSS rises to the top.
- **Confidence next:** confirmed beats unverified.
- **Access value:** does it likely grant a foothold or admin?

> "What I'd try first, and why" — that's the plan, nothing more.

Lab Part E — CVE mapping + attack plan

1. **Map** 2–3 starred versions to their CVE(s) and CVSS.
2. Fill the CVE-mapping table; flag one likely **false positive** + propose verification.
3. **Draft a prioritized attack plan** — what an attacker would try first and why.

> **Do not exploit anything.** We stop at the plan.

Check your understanding

> Two confirmed findings: an FTP backdoor (CVSS ~10) and an outdated web library (CVSS 5.3).

Which goes first in your attack plan, and why?

Think before the next slide.

Answer

The **FTP backdoor** goes first.

- **Higher CVSS** (~10 vs 5.3) → more severe.
- A backdoor likely grants a **direct foothold**.
- Both are confirmed, so severity + access value decide order.

Exit ticket — Day 4

What is a **false positive**, and how would you **verify** a scanner finding before trusting it?

Day 5

Recon mini-project (Module 2 milestone)

Warm-up & ethics re-anchor

> We can build an attack plan — why do we **STOP** before exploiting?

- **Finding a vuln is not permission to exploit it.**
- Exploitation is a separate, more serious act needing its own authorization.
- In this course, exploitation waits for **Module 3** on authorized targets.

The Recon mini-project (Module 2 milestone)

The capstone of the whole module — a professional **Recon Report** combining:

- **Unit 07** — passive recon context/awareness
- **Unit 08** — active scanning results
- **Unit 09** — enumeration, vuln findings, CVE mappings, attack plan

> A graded **Project** (25% category). Get the rubric **before** you start.

Recon report — structure

1. **Scope statement** confirming authorization (planning only)
2. **Methodology** — recon → scan → enumerate → vuln-map
3. **Findings** — each with evidence, version/CVE, severity
4. **Recommended remediations** — specific fixes per finding

> One professional document that tells the whole story.

Writing a strong finding

Each finding should include:

- **What** it is + **evidence** (a scan line, a banner).
- The **version** and **CVE**, with **CVSS** severity.
- **Why it matters** — the risk in plain language.

> Evidence + severity + impact = a finding a reader can trust.

Findings → remediations

Finding	Remediation
vsftpd 2.3.4 (backdoored)	Upgrade/replace FTP server
Anonymous FTP allowed	Disable anonymous login
Open SMB shares + users	Restrict shares, require auth
Old OpenSSH version	Patch to a supported release

Every finding gets a fix. That's what makes it a **report**, not a brag.

Lab — assemble the mini-project

- Open your Unit 08 annotated scans and Parts A–E findings.
- Build the multi-section **Recon Report** on the **authorized lab target**.
- Include the scope statement; keep it **planning only**.

> This report becomes the launch point for **Module 3** and the final pentest report (Unit 17).

Ethics & Authorization

Enumeration and vuln scanning are ACTIVE — and louder than a port scan.

They send probes and sometimes **login attempts** to the target. **All of it requires authorization.**

Finding a vuln is NOT permission to exploit

- A scanner saying *"critical vulnerability"* is a **finding**, not a green light.
- Finding a weakness and attacking it are **completely different acts** — legally and ethically.
- Real, unauthorized finding? Follow **responsible disclosure** (Unit 01).

> We stay on the **authorized lab target** and **stop at the plan.**

Responsible disclosure in one slide

If you find a real vuln on a system you don't own:

- **Don't** exploit it.
- **Don't** post it publicly.
- **Do** report it privately to the owner.

> Quiet, ethical, and legal — in that order.

Discussion

You scan a friend's website **without asking** and a tool reports a critical vulnerability.
What are your **legal and ethical obligations**? What should you do — and what must you **absolutely NOT** do?

Recap

- **Scanning → enumeration → vuln scanning** — each digs deeper.
- Manual enumeration finds context (users, shares) tools miss.
- **NSE** and automated scanners speed things up — but produce **false positives**.
- **CVE + CVSS** turn a version string into a measured risk.
- **Finding a vuln is not permission to exploit it.** Stop at the plan.
- Everything stays on the **authorized lab target**.

Key vocabulary — quick review (1 of 2)

Term	Meaning
Enumeration	Digging into a service for users/shares/versions
Vuln scanning	Matching services to known weaknesses
SMB	Windows file/printer sharing (often enumerable)
NSE	Nmap Scripting Engine

Key vocabulary — quick review (2 of 2)

Term	Meaning
CVE	Public ID for one known vulnerability
CVSS	0–10 severity score
False positive	A reported "vuln" that isn't real
Attack plan	Prioritized list of what to try, and why

Exit ticket — Day 5 + homework

Exit ticket: one sentence on responsible disclosure.

Homework:

- Finish the Recon mini-project report draft.
- ½-page: "A scanner flags a 'critical' vuln on a server you don't own. What do you do and what must you NOT do?" Use **authorization** and **responsible disclosure**.

Module 2 complete. Next: Module 3 — Exploitation, on authorized targets only.